

ТЗ на лабораторную работу: Разработка CLI-аналитика фильмов

Источник данных: Документация API poiskkino.dev.

Предоставленный API-токен: DP7GYZK-T5W4855-GSS2A69-RJFZ80F (передается в заголовке X-API-KEY).

ЧАСТЬ 1. СОВМЕСТНАЯ РАБОТА

Тема: Выполнение HTTP-запросов, авторизация по токену, обработка ошибок сети и сериализация JSON.

Шаг 1.1. Инициализация и константы

В файле main.py определить базовые настройки приложения:

- API_URL
- TOKEN

Шаг 1.2. Реализация сетевого взаимодействия

Написать функцию `fetch_movies_data(limit=30)`, выполняющую синхронный GET-запрос.

Параметры конфигурации запроса:

1. Заголовки (Headers)

2. Параметры строки запроса:

- `limit: 30` (запрашиваем 30 фильмов для репрезентативной аналитики).
- `page: 1`
- `selectFields: ["id", "name", "year", "rating", "genres", "countries", "type"]` (ограничиваем выборку для экономии трафика).
- `notNullFields: ["name", "rating.kp", "genres.name"]` (исключаем из выдачи тайтлы с незаполненными ключевыми данными).
- `sortField: ["rating.kp"]`
- `sortType: ["-1"]` (сортировка по убыванию рейтинга - получаем топ-фильмы).

- `type: ["movie"]` (выбираем только полнометражные фильмы, исключая сериалы и мультфильмы).

Шаг 1.3. Валидация ответа сервера

1. Реализовать проверку `response.status_code`. Если код ответа не равен 200, возбудить исключение или корректно завершить работу скрипта, выведя код ошибки.
2. С помощью метода `response.json()` преобразовать тело ответа в Python-словарь.

Шаг 1.4. Локальное кэширование данных

1. Создать функцию `cache_data_to_json(data_dict, filename="movies_cache.json")`.
2. Функция должна открыть файл на запись в режиме `w` с явным указанием кодировки `encoding="utf-8"`.
3. Сохранить объект с помощью `json.dump()` со следующими флагами: `ensure_ascii=False` (для сохранения кириллицы) и `indent=4` (для форматирования отступов).

ЧАСТЬ 2. САМОСТОЯТЕЛЬНАЯ РАБОТА

Тема: Десериализация локальных файлов, алгоритмический анализ структур данных и запись форматированного текста.

Ограничение этапа: Импорт модуля `requests` и выполнение любых запросов в интернет на этой паре строго **запрещены**. Все вычисления производятся локально на основе файла `movies_cache.json`.

Шаг 2.1. Чтение кэшированных данных

1. Написать функцию `load_cached_movies(filename="movies_cache.json")`.
2. Функция считывает данные через `json.load()`.
3. **Обязательное требование:** Обернуть блок чтения в конструкцию `try-except` для перехвата ошибки `FileNotFoundError`. Если файл

отсутствует, программа не должна аварийно завершаться; требуется вывести сообщение: *«Ошибка: Файл кэша отсутствует. Сначала запустите скрипт из Части 1»*.

Шаг 2.2. Разработка аналитического движка

Необходимо обработать массив фильмов, находящийся по пути `data["docs"]`, выделив и рассчитав следующие метрики:

1. **Метрика количества:** Посчитать фактическое количество записей в массиве.
2. **Метрика экстремумов рейтинга:** Найти фильм с самым высоким рейтингом (`rating["kp"]`) и с самым низким в данной выборке. Вывести их названия и оценку.
3. **Хронологический анализ:** Найти самый старый фильм (минимальное значение поля `year`).
4. **Агрегация жанров (Частотный анализ):** Пройтись по списку `genres` каждого фильма (внутри объекта это массив видов `[{"name": "драма"}, {"name": "криминал"}]`). Подсчитать суммарное количество упоминаний каждого жанра во всей выборке. Определить **Топ-3 самых популярных жанров**.

Шаг 2.3. Генерация отчетной документации

1. Создать функцию `write_analytics_report(metrics_dict, filename="movies_report.txt")`.
2. Функция собирает все вычисленные метрики в единый строковый блок. Данные должны быть отформатированы с использованием f-строк.
3. Записать данные в файл `movies_report.txt` в кодировке `utf-8`. Текст должен быть структурирован (применяйте визуальные разделители линий вроде `---` или `===`).

Шаблон ожидаемого отчета `movies_report.txt`:

=====

АНАЛИТИЧЕСКИЙ ОТЧЕТ ПО ДАННЫМ ROISKKINO.DEV

=====

Успешно обработано тайтлов: 30

1. РЕЙТИНГОВЫЕ ПОКАЗАТЕЛИ:

- Лидер рейтинга: [Название фильма] (Рейтинг КР: [X.XX])
- Аутсайдер выборки: [Название фильма] (Рейтинг КР: [X.XX])

2. ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ:

- Самый старый фильм в кэше: [Название] ([XXXX] год)

3. СТАТИСТИКА ПО ЖАНРАМ (ТОП-3):

1. [Жанр_1]: [Количество] фильмов
 2. [Жанр_2]: [Количество] фильмов
 3. [Жанр_3]: [Количество] фильмов
- =====